

Actividad de laboratorio: introducción práctica a sistemas P2P

Laboratorio TIC — DIINF

Curso: Sistemas Distribuidos y Paralelos
Departamento de Ingeniería Informática — USACH

10 de julio de 2026

1. Objetivo de aprendizaje

En una hora, cada grupo montará una mini-red P2P entre sus nodos, repartirá un archivo en piezas, implementará una estrategia de descarga y medirá tiempos y mensajes ante fallas. El foco está en **experimentar y medir**, no solo en revisar teoría.

2. Rol del docente

El docente orienta con **teoría** y responde dudas si las solicitan. **No** prepara archivos ni configura la red: eso lo hace cada grupo.

3. Repositorio y fork

Código base y este enunciado están en el repositorio público del curso:

https://github.com/miguelcarcamov/lab_experiences

Cada grupo debe:

1. Hacer **fork** del repositorio a su cuenta de GitHub.
2. Clonar **su fork** en al menos una máquina del grupo:

```
git clone git@github.com:<usuario>/lab_experiences.git
cd lab_experiences/p2p-intro
```

3. Crear una rama de trabajo, por ejemplo `grupo-2/p2p-intro`.
4. Implementar `select_next_piece` en `node.py` y commitear en esa rama.
5. No subir artefactos generados en el lab (`demo.bin`, `payload/`, `piezas/`); el `.gitignore` ya los excluye.

4. Organización

- **Lugar:** Laboratorio TIC, DIINF.
- **Duración:** 60 minutos.
- **Grupos:** 3 equipos de 5 estudiantes (los mismos del laboratorio del curso).
- **Modalidad:** cada grupo usa varios nodos del lab (recomendado: al menos 3 por grupo). Deben conectarlos en la misma red local (switch) y anotar la IP de cada máquina.

5. Materiales y prerequisites

- Nodos del laboratorio TIC y switch para interconectarlos.
- Python 3 (solo librería estándar).
- Carpeta `p2p-intro/` del fork (archivos principales):
 - `node.py` — servidor, protocolo y `fetch` listos; **TODO:** `select_next_piece(...)`.
 - `discover.py` — descubrimiento TCP, listo para usar.
 - `prepare_payload.py` — generan y trocean el archivo de prueba del grupo.
 - `activity_1.pdf` — este enunciado.
- Papel para dibujar el grafo de conectividad.

5.1. Preparación del archivo (lo hace el grupo)

1. Generar un archivo de prueba: `python3 prepare_payload.py --gen-demo` (crea `demo.bin`, ~48 KB).
2. Trocearlo y repartir piezas entre los nodos del grupo, por ejemplo con 4 nodos:

```
python3 prepare_payload.py --input demo.bin --chunk-size 4096 --nodes 4
```

Esto produce `manifest.json` y carpetas `payload/nodes/node01/`, ... con subconjuntos de `chunk_NNN.bin`.

3. Copiar en cada PC su carpeta como `./piezas/` (cada nodo arranca con **parte** del archivo, nadie con el archivo completo).
4. Crear `seeds.txt` y `peers.txt` con las IP reales del grupo (`host:8800:node-id` por línea).

La reconstrucción es correcta si `recuperado.bin` tiene el mismo SHA-256 que `manifest.json`.

5.2. Por qué el ejercicio es `select_next_piece`

La plomería de red ya está resuelta en `node.py`. Lo pedagógico es **decidir a qué peer pedir qué pieza** (secuencial, *rarest-first*, aleatorio, etc.): eso cambia mensajes, tiempos y tolerancia a fallas.

6. Desarrollo de la actividad

Tiempo	Fase	Qué hace cada grupo
0–10 min	Fork + red + descubrimiento	Fork y clone del repo; conectar nodos; <code>prepare_payload.py</code> ; copiar piezas/; <code>node.py</code> <code>serve</code> ; <code>seeds.txt/peers.txt</code> ; <code>discover.py</code> y grafo.
10–40 min	Implementación + difusión	10–15 min: completar <code>select_next_piece</code> y commit en la rama del grupo. 15–40 min: <code>node.py</code> <code>fetch</code> ...; anotar tiempos y mensajes.
40–50 min	Inyección de falla	Otro grupo detiene 1–2 nodos <code>serve</code> sin aviso. Reintentar <code>fetch</code> ; anotar detección y recuperación.
50–60 min	Cierre con datos	Plenario: estrategia, tiempos, mensajes, grafo, efecto de la falla. URL del fork si el docente lo pide.

6.1. Indicaciones operativas

- Repartan roles (red, payload, implementación, Git, medición, reporte).
- Solo modifiquen `select_next_piece` en `node.py` salvo acuerdo documentado del grupo.
- Si `discover.py` no ve peers, revisen IP, firewall y que `serve` esté activo.
- Documenten la estrategia usada en la medición final.

7. Preguntas de reflexión (cierre grupal)

1. ¿Qué estrategia implementaron en `select_next_piece`? ¿Por qué?
2. ¿Cómo detectaron un nodo caído?
3. ¿Cuántos mensajes y cuánto tiempo antes y después de la falla?
4. Si cayeran la mitad de los nodos, ¿su estrategia seguiría funcionando? ¿Qué faltaría (DHT, réplicas, *rarest-first* global)?

8. Extensión opcional

Quienes terminen antes pueden probar *rarest-first* o esbozar cómo localizarían piezas sin `peers.txt` central.

Entregable mínimo (si aplica): URL del fork, rama usada, nodos e IPs, estrategia, tiempos y mensajes, reacción ante la falla, grafo de conectividad.